

Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System

CO3-ASSESSMENT 1

Course:

Software Engineering / Code Review and Version Control Evaluation

Name:

Davis Damien M

192521124

Problem Analysis

Problem statement

Hotels manage reservations, room allocation, housekeeping, maintenance, guest requests, billing, and customer support through multiple disconnected systems. Manual coordination causes delays, overbooking, poor guest experiences, inefficient staff utilization, and limited operational insight. An AI-based smart hotel operations and guest experience management system integrates these functions into a single intelligent platform that automates operations and personalizes guest services using machine learning and analytics.

Project objectives

- Automate hotel operational workflows.
- Provide AI-driven guest recommendations and personalized services.
- Predict room occupancy and optimize resource allocation.
- Enable real-time housekeeping and maintenance management.
- Improve guest satisfaction through a mobile/web interface.
- Generate analytics dashboards for hotel administrators.

Functional requirements

- User authentication (guest, staff, admin).
- Room booking and availability management.
- AI recommendation engine (room upgrades, dining, amenities).
- Housekeeping task assignment and tracking.
- Maintenance request management.
- Billing and payment processing.
- Notifications (email/SMS/app).
- Admin dashboard with reports and occupancy analytics.

Expected outcomes

- Reduced operational workload.
- Faster guest request handling.
- Improved occupancy forecasting.
- Higher guest satisfaction.

- Scalable and portable deployment using Docker.
- Collaborative development using Git.

2. Project Structure Design

Repository folder structure

```
hotel-ai-system/  
├── backend/  
│   ├── app/  
│   ├── models/  
│   ├── routes/  
│   ├── services/  
│   ├── ai/  
│   ├── requirements.txt  
│   └── Dockerfile  
├── frontend/  
│   ├── src/  
│   ├── public/  
│   ├── package.json  
│   └── Dockerfile  
├── database/  
│   └── init.sql  
├── docker-compose.yml  
├── .env  
├── .gitignore  
├── README.md  
├── docs/  
│   ├── architecture.png  
│   ├── workflow.png  
│   └── report.pdf  
└── tests/  
    ├── backend/  
    └── frontend/
```

Purpose of major folders and files

- backend/: API, authentication, business logic, AI services.
- frontend/: Guest and admin web interface.

- database/: Database initialization scripts.
- docs/: Architecture diagrams and documentation.
- tests/: Automated unit and integration tests.
- docker-compose.yml: Multi-container orchestration.
- .env: Environment configuration.
- .gitignore: Excludes unnecessary files from version control.
- README.md: Project overview and setup instructions.

3. Git Repository Initialization

Steps

```
git init
git add .
git commit -m "Initial project structure"
git branch -M main
git remote add origin https://github.com/username/hotel-ai-system.git
git push -u origin main
```

Essential Git files

- .git/: Internal repository metadata.
- .gitignore: Prevents committing build files, virtual environments, logs, and secrets.
- README.md: Documents project purpose and usage.

4. Git Branching Strategy

Branch model

```
main
├── develop
│   ├── feature-auth
│   ├── feature-booking
│   ├── feature-ai-recommendation
│   ├── feature-housekeeping
│   └── feature-dashboard
```

└─ release/v1.0
└─ hotfix/login-bug

Justification

- main: Stable production-ready code.
- develop: Integration branch for ongoing development.
- feature/*: Isolated implementation of individual modules.
- release/*: Final testing before production deployment.
- hotfix/*: Urgent production bug fixes.

This model supports parallel team development while protecting production stability.

5. Version Control Workflow

Example workflow

```
git checkout -b feature-booking  
git add backend/routes/booking.py  
git commit -m "Add room booking API"
```

```
git checkout develop  
git merge feature-booking
```

```
git checkout -b feature-ai-recommendation  
git add backend/ai/recommendation.py  
git commit -m "Implement AI room recommendation service"
```

```
git checkout develop  
git merge feature-ai-recommendation
```

```
git checkout main  
git merge develop
```

```
git pull origin main  
git push origin main
```

Conflict resolution

```
git status  
git add resolved_file.py  
git commit -m "Resolve merge conflict in booking service"
```

Purpose:

- Branch creation: Isolates development work.
- Commit: Saves logical changes.
- Merge: Integrates completed features.
- Conflict resolution: Reconciles concurrent modifications.
- Pull/push: Synchronizes local and remote repositories.

6. Commit History Analysis

Example commit history

a1b2c3d Initial project structure
 b2c3d4 Add authentication module
 c3d4e5 Implement room booking API
 d4e5f6 Integrate PostgreSQL database
 e5f6g7 Add AI recommendation engine
 f6g7h8 Implement housekeeping management
 g7h8i9 Create admin dashboard
 h8i9j0 Add Docker configuration
 i9j0k1 Resolve booking merge conflict
 j0k1l2 Prepare release v1.0

Good commit practices

- Uses imperative mood (Add, Implement, Integrate).
- One logical change per commit.
- Clear and descriptive messages.
- Facilitates debugging and code review.
- Improves project maintainability and traceability.

7. Docker Environment Design

Why Docker is suitable

- Consistent development and production environments.
- Eliminates dependency conflicts.
- Simplifies deployment.
- Supports microservice architecture.

- Enables rapid scaling.

Components requiring containerization

- Backend API service.
- Frontend web application.
- PostgreSQL database.
- Redis cache (optional).
- AI inference service.

8. Dockerfile Development

Backend Dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main", "--host", "0.0.0.0", "--port", "8000"]

Instruction explanation

- FROM: Base Python image.
- WORKDIR: Sets working directory.
- COPY: Copies dependency and source files.
- RUN: Installs Python packages.
- EXPOSE: Documents application port.
- CMD: Starts the FastAPI server.

9. Docker Compose Design

docker-compose.yml

version: '3.9'

```
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgres://hotel@db:5432/hoteldb
    depends_on:
      - db

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: hotel
      POSTGRES_PASSWORD: password
      POSTGRES_DB: hoteldb
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Explanation

- backend: REST API service.
- frontend: Web interface.
- db: PostgreSQL database.
- depends_on: Ensures startup order.
- volumes: Persistent database storage.
- environment: Database configuration.
- Networking: Docker Compose creates a shared bridge network.

10. Container Deployment and Testing

Deployment

`docker-compose build`

`docker-compose up -d`

Verification

`docker ps`

Expected containers:

- hotel-backend
- hotel-frontend
- hotel-db

Testing procedure

- Open <http://localhost:3000>.
- Log in as admin.
- Create a room booking.
- Verify booking appears in the dashboard.
- Check backend API:
`curl http://localhost:8000/api/health`
- Inspect database records using PostgreSQL client.

Expected result: All services communicate successfully and booking operations complete without errors.

11. Benefits of Git and Docker

Git benefits

- Parallel team collaboration.
- Complete version history.
- Easy rollback.
- Branch-based feature development.
- Code review through pull requests.
- Traceability of changes.

Docker benefits

- Environment consistency.
- Portable deployment across systems.
- Faster onboarding.
- Simplified dependency management.
- Horizontal scalability.
- Easier maintenance and updates.

Combined, Git and Docker provide reliable CI/CD-ready development and deployment workflows.

12. Challenges and Improvements

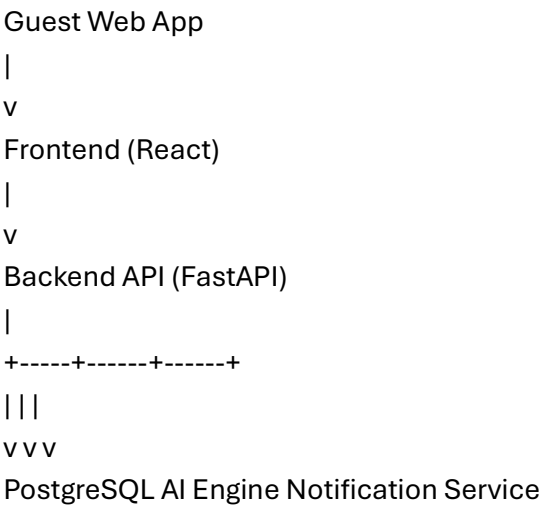
Challenges encountered

- Merge conflicts during concurrent development.
- Docker networking configuration.
- Environment variable management.
- Database initialization timing.
- Large Docker image sizes.

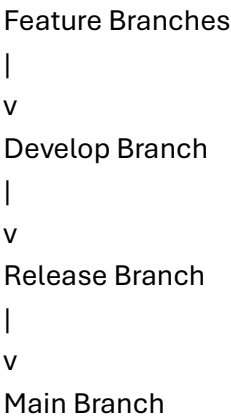
Improvements

- Use GitHub Flow with protected branches.
- Add pull request templates and code reviews.
- Integrate GitHub Actions CI/CD.
- Use Docker multi-stage builds.
- Add Nginx reverse proxy.
- Implement Kubernetes deployment.
- Add Redis caching.
- Integrate monitoring with Prometheus and Grafana.
- Implement automated backup and disaster recovery.

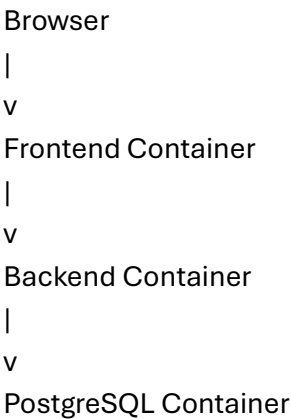
Architecture Diagram



Git Workflow Diagram



Docker Deployment Diagram



Conclusion

The Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System demonstrates how Git enables structured collaborative software development while Docker provides portable and consistent deployment across environments. The project architecture supports AI-powered hotel automation, scalable microservice deployment, and maintainable version-controlled development suitable for modern hospitality management systems.